

EcoRover: A Mobile IoT Multi-Sensor Platform for Environmental Risk Mapping in Smart Campus Scenarios

Jordan Jessid Del Aguila Ruiz² , Jesús Alejandro Espinosa Gutiérrez³ , Dayron Camilo Echeverry Cortes¹ , Hayleen Yaffrianny Mosquera Mosquera¹, Juan Carlos Morales Guerra² , and Jorge Andres Montoya Cardona¹, *

- ¹ Department of Electronics and Telecommunications, Faculty of Engineering, Institución Universitaria ITM, Medellín, Colombia; dayronecheverry1136631@correo.itm.edu.co (D.C.E.C.); hayleenmosquera1137865@correo.itm.edu.co (H.Y.M.M.); jorgemontoyac@itm.edu.co (J.A.M.C.)
- ² Department of Systems, Faculty of Engineering, Institución Universitaria ITM, Medellín, Colombia; jordandel1151574@correo.itm.edu.co (J.D.R.); juanmorales@itm.edu.co (J.C.M.G.)
- ³ Department of Systems and Computing, Computer Systems Engineering Program, Instituto Tecnológico de Morelia, Morelia, Michoacán, Mexico; 123120509@morelia.tecnm.mx (J.A.E.G.)
- * Correspondence: jorgemontoyac@itm.edu.co
- † Presented at the 13th International Electronic Conference on Sensors and Applications, 18–20 November 2026; Available online: <https://sciforum.net/event/ECSA-13>.

Abstract

Fixed environmental monitoring nodes provide continuous observations but limited spatial coverage. This paper presents EcoRover, a low-cost mobile Internet of Things (IoT) prototype developed by students for exploratory monitoring in smart-campus environments. The platform integrates multi-sensor acquisition, reactive navigation, local storage, GPS positioning, HTTP communication with ThingSpeak, and a React web dashboard. Functional tests supported the integration of its sensing, mobility, storage, communication, and visualization functions while guiding the replacement of unstable components. The prototype established an end-to-end path from data acquisition to remote consultation and maintained local logging without network connectivity. EcoRover provides a practical platform for education and future georeferenced studies. Calibration, controlled monitoring routes, and quantitative performance evaluation are required before the platform can generate defensible environmental risk maps.

Keywords: Mobile IoT; Environmental monitoring; Smart campus; Sensor fusion; Educational robotics

1. Introduction

University campuses combine academic buildings, laboratories, roads, green areas, pedestrian corridors, and semi-outdoor spaces within a bounded environment. This diversity makes them useful test beds for smart-city technologies and for evaluating services before broader urban deployment. The smart-campus concept applies sensing, communication, data processing, and connected infrastructure to support operational management, sustainability, teaching, and research [1,2]. Within this context, the Internet of Things (IoT) enables heterogeneous devices to collect measurements and exchange them with remote services, providing the technical basis for distributed monitoring and visualization [3].

Environmental monitoring is one of the most direct applications of this infrastructure because temperature, humidity, particulate matter, illumination, and acoustic conditions

Published:

Copyright: © 2026 by the authors.

Submitted to *Eng. Proc.* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

may vary among indoor, outdoor, and transitional campus spaces. Conventional reference-grade stations provide high-quality measurements, but their cost and fixed installation limit dense spatial coverage. Portable low-cost sensors can complement these stations by increasing measurement density and enabling exploratory observations in locations that would otherwise remain unmonitored [4,5]. Their reduced cost, however, does not guarantee metrological quality. Sensor response can be affected by environmental conditions, aging, unit-to-unit variability, installation, and the selected calibration procedure. Consequently, data from low-cost devices must be interpreted according to their intended use and should not be presented as regulatory measurements without appropriate calibration and comparison with reference instrumentation [5].

Mobility can extend the spatial reach of a sensing payload without requiring a permanent node at every location. Environmental robots have been used to transport instruments, adapt sampling locations, and collect observations in terrestrial, aquatic, and aerial applications [6]. In built environments, sensor-rich mobile robots have also been investigated as an alternative to dense fixed networks for measuring indoor environmental quality [7]. Nevertheless, mobile sensing introduces methodological challenges because each reading is associated with both position and time. Studies on urban air-quality monitoring show that representative spatial maps require carefully designed routes, repeated measurements, temporal normalization, and explicit data-quality procedures [8,9]. A mobile prototype may therefore demonstrate acquisition, localization, communication, and visualization functions without yet constituting a validated environmental mapping system.

For university projects, the value of such a platform also lies in the integration process. A mobile IoT system combines embedded programming, sensor interfaces, electronics, wireless communication, data management, web development, and robotic control within a single experimental artifact. Educational-robotics research indicates that construction and programming activities can support learning in science, technology, engineering, and problem solving, although their effectiveness depends on the instructional design and the evidence used for evaluation [10]. A platform intended for this context should therefore expose its architecture and implementation decisions clearly, while separating demonstrated functions from capabilities that require future experimental validation.

This work presents EcoRover, a low-cost mobile IoT multi-sensor platform built by students at Institución Universitaria ITM for environmental monitoring research and education in smart-campus scenarios. The prototype combines an environmental sensing subsystem with a separate reactive navigation subsystem. The sensing firmware, implemented in MicroPython on an ESP32, coordinates temperature and humidity, particulate matter, light, relative acoustic activity, rain, color, proximity, and Global Positioning System (GPS) modules; it also provides local liquid-crystal display (LCD) feedback and comma-separated-value (CSV) logging. A second control branch uses three ultrasonic sensors, an L298N driver, and two motors to perform obstacle-aware movement. Selected measurements and GPS data are sent to ThingSpeak through Hypertext Transfer Protocol (HTTP) GET requests and are consulted through a React-based dashboard.

The contribution of this paper is the design and functional integration of EcoRover as a student-built platform rather than the validation of a completed environmental-risk map. Specifically, the work documents: (i) a modular hardware architecture that combines heterogeneous environmental and navigation sensors; (ii) object-oriented sensing firmware with scheduled acquisition, local display, GPS processing, and CSV storage; (iii) reactive obstacle avoidance implemented on the mobile base; and (iv) an HTTP-to-ThingSpeak data pipeline connected to four dashboard views for current, historical, and geographic visualization. The current study does not report a calibrated measurement campaign, predefined monitoring routes, hotspot analysis, or a validated risk index. Instead, it

establishes an experimentally implemented foundation for future georeferenced campaigns after sensor calibration and a reproducible sampling protocol have been defined.

2. EcoRover System Design and Hardware Implementation

This section describes the functional organization and hardware implementation of EcoRover. The system architecture is presented first, followed by the electronic components, communication interfaces, and physical integration of the sensing and navigation subsystems on the mobile platform.

2.1. System Architecture

EcoRover was implemented using two ESP32-based control nodes integrated into the same mobile platform: a sensing and IoT node and a navigation node. This division separates environmental data acquisition, local storage, and wireless communication from obstacle detection and motor actuation. Figure 1 presents the resulting hardware architecture and the connections between the controllers, sensors, local interfaces, actuators, and remote monitoring service.

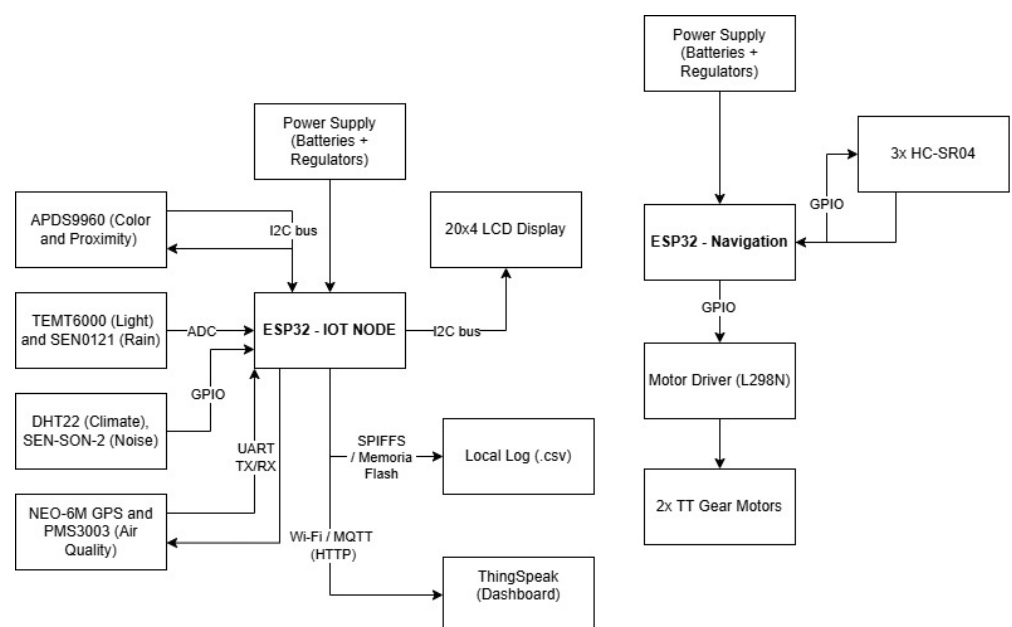


Figure 1. Hardware architecture of the EcoRover prototype, showing the sensing and IoT node and the navigation subsystem.

The sensing and IoT node, shown on the left side of Figure 1, is responsible for coordinating the environmental and positioning sensors. The DHT22 measures temperature and relative humidity, while the PMS3003 provides particulate-matter readings for PM1.0, PM2.5, and PM10. The TEMT6000 and SEN0121 modules are connected through analog inputs and provide relative light-intensity and surface-water measurements, respectively. The SEN-SON-02 supplies a digital signal used to estimate relative acoustic activity, and the APDS9960 provides proximity and RGB color information through the I²C bus. Geographic positioning is obtained from the NEO-6M GPS module through a UART connection.

The same ESP32 controls a 20×4 LCD that presents the acquired variables locally. Measurements are also stored periodically in a CSV file in the controller's internal flash memory. When Wi-Fi connectivity is available, selected environmental variables and GPS coordinates are transmitted to ThingSpeak through HTTP GET requests. ThingSpeak therefore acts as the remote data service used by the web dashboard, while local acquisition and CSV storage remain available independently of cloud transmission.

The navigation subsystem, shown on the right side of Figure 1, operates through a second ESP32 running dedicated movement-control firmware. Three HC-SR04 ultrasonic sensors measure the distances in front of and on both sides of the rover. Based on these measurements, the controller sends direction commands to an L298N motor driver, which operates the two TT geared motors. This subsystem implements reactive obstacle avoidance based on the measured distances; it does not depend on predefined routes, stored waypoints, or commands received from the ThingSpeak dashboard.

The two control nodes perform complementary functions on the same chassis. The sensing node manages environmental acquisition and data communication, whereas the navigation node maintains the rover's movement and responds to nearby obstacles. This functional separation allowed both subsystems to be developed and tested independently before their physical integration into the final prototype.

2.2. Hardware Components

The hardware selection and integration process began with the individual evaluation of the sensors and peripheral modules considered for EcoRover. Each device was tested separately to verify its electrical requirements, communication interface, and ability to provide readable data before being incorporated into the complete platform. Only the components that operated consistently during these preliminary tests were retained in the final configuration. Table 1 summarizes the electrical and functional characteristics documented for the components used in the implemented prototype.

The environmental sensing node combines devices with different interfaces and types of output. The DHT22 provides temperature and relative-humidity readings, while the PMS3003 communicates through UART and reports particulate-matter values for PM1.0, PM2.5, and PM10. The TEMT6000 and SEN0121 are connected to analog inputs and provide relative responses associated with incident light and water on the sensing surface, respectively. Because these analog modules were not calibrated against reference instruments, their outputs are treated as relative indicators rather than measurements expressed in standardized illumination or precipitation units.

The SEN-SON-02 provides a digital signal whose activation threshold can be adjusted through an onboard potentiometer. The sensing firmware counts its activations within a defined time window to obtain a relative indicator of acoustic activity; consequently, its output is not interpreted as a sound-pressure level in decibels. The APDS9960 complements the environmental inputs by providing proximity and RGB color information through the I²C bus.

Table 1. Electrical and functional characteristics of the components used in the EcoRover prototype.

Component	Operating voltage	Documented specification or response	Measured variable or function
DHT22	3.3 V	Temperature accuracy: $\pm 0.5^{\circ}\text{C}$; relative-humidity accuracy: $\pm 2\text{--}5\%$ RH	Ambient temperature and relative humidity
PMS3003	5 V	Documented tolerance of $\pm 10\ \mu\text{g m}^{-3}$ within the 0–100 $\mu\text{g m}^{-3}$ range	PM1.0, PM2.5, and PM10 particulate-matter readings
TEMT6000	3.3 V	Analog response varying proportionally with incident light intensity	Relative ambient-light level
SEN0121	3.3 V	Analog response varying with the amount of water on the sensing surface	Relative surface-water or rain indication
SEN-SON-02	3.3 V	Digital output with an adjustable activation threshold	Relative acoustic-activity indicator
APDS9960	3.3 V	Digital proximity and red, green, and blue response channels	Proximity and RGB color information
NEO-6M GPS	5 V	Typical horizontal-position accuracy of approximately 2.5 m under suitable reception conditions	Latitude, longitude, positioning status, and number of satellites
HC-SR04 (3 units)	5 V	Nominal ranging accuracy of $\pm 0.3\text{ cm}$	Front, right, and left obstacle-distance measurements
20×4 LCD	5 V	I ² C interface	Local visualization of sensor measurements and system status
L298N motor driver	5–35 V	Bidirectional control of two DC motor channels	Direction control and actuation of the traction motors
TT geared motors (2 units)	3–6 V	Speed and torque depend on the applied voltage and mechanical load	Rover propulsion and turning maneuvers
ESP32 development boards (2 units)	3.0–3.6 V	Clock frequency of up to 240 MHz, with digital, analog, UART, I ² C, and Wi-Fi interfaces	Processing of the sensing and navigation sub-systems

The listed values correspond to nominal or documented component specifications used as design references. They do not represent calibration or metrological-validation results obtained in this study.

The NEO-6M GPS module is connected through UART and provides geographic coordinates when a valid satellite solution is available. GPS operation is therefore intended for open or sufficiently exposed areas and may be limited near buildings or covered campus spaces. A 20×4 LCD shares the I²C bus with the APDS9960 and provides local feedback through six display pages containing environmental measurements, particulate-matter readings, and positioning status.

The navigation hardware consists of a second ESP32, three HC-SR04 ultrasonic sensors, an L298N motor driver, and two TT geared motors. The ultrasonic sensors are oriented toward the front and sides of the rover to support reactive obstacle detection. The navigation controller processes these distances and activates the motor driver to move forward, stop,

or perform corrective turns. Motor operation is implemented at a fixed speed in the current firmware because pulse-width modulation is not used.

Additional support elements include battery and voltage-regulation stages, external pull-up resistors for the shared I²C bus, wiring, and a perforated phenolic board used for the permanent assembly. The pull-up resistors and differentiated update intervals for the LCD and APDS9960 were introduced after communication losses and timeout errors were observed during integration. Once the individual devices had been tested, the sensing electronics were transferred from the initial breadboard assembly to the perforated board and mounted on the mobile chassis.

The BME680 and KY-038 were evaluated during the initial component-testing stage but were not retained in the final EcoRover configuration. Problems were observed with the tested BME680 and with several KY-038 units. The DHT22 was therefore used for temperature and relative-humidity acquisition, while the SEN-SON-02 was incorporated as the final acoustic-activity sensor. Accordingly, neither device forms part of the implemented EcoRover configuration.

2.3. Prototype Assembly

The final physical implementation of EcoRover is presented in Figure 2. The upper platform contains the sensing board and exposes the environmental modules to the surrounding conditions, while the LCD is positioned at the rear for local inspection. The three ultrasonic sensors are oriented toward the front and sides of the chassis, and the traction, power, and motor-control elements are located on the lower level. This arrangement keeps the sensing surfaces and obstacle-detection modules unobstructed while preserving access to the electronics during experimental testing.

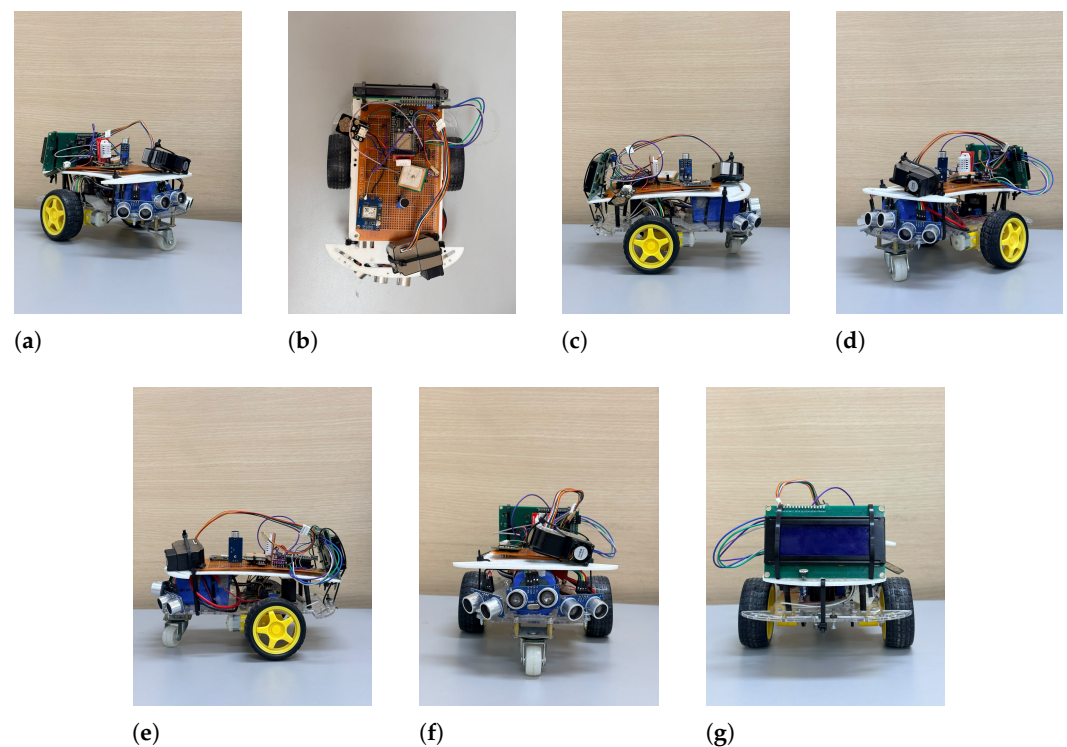


Figure 2. Assembled EcoRover prototype: (a) and (d) oblique front views from opposite sides; (b) top view; (c) and (e) lateral views; (f) front view showing the three ultrasonic sensors; and (g) rear view showing the LCD interface.

3. Firmware, Data Acquisition, and IoT Monitoring

This section describes the software developed to coordinate sensor acquisition, local data management, reactive navigation, and remote IoT monitoring. The sensing and navigation routines are implemented as separate firmware modules, while the dashboard uses the information transmitted by the sensing node for remote visualization.

3.1. Sensor Acquisition and Local Data Management

The sensing firmware was developed in MicroPython using a modular object-oriented structure. The file `sensores.py` contains the classes responsible for initializing and reading the individual sensing devices, while `main.py` coordinates their execution, the LCD interface, local storage, Wi-Fi connectivity, and transmission to ThingSpeak. Sensor results are returned as key–value dictionaries, providing a consistent mechanism for exchanging data between the sensor classes and the main program.

Figure 3 summarizes the execution flow of the sensing firmware. A relevant aspect of this sequence is that the local logging operation precedes the connectivity decision. Therefore, when Wi-Fi is unavailable, the cloud request is skipped but the sensor record can still be written to the local file.

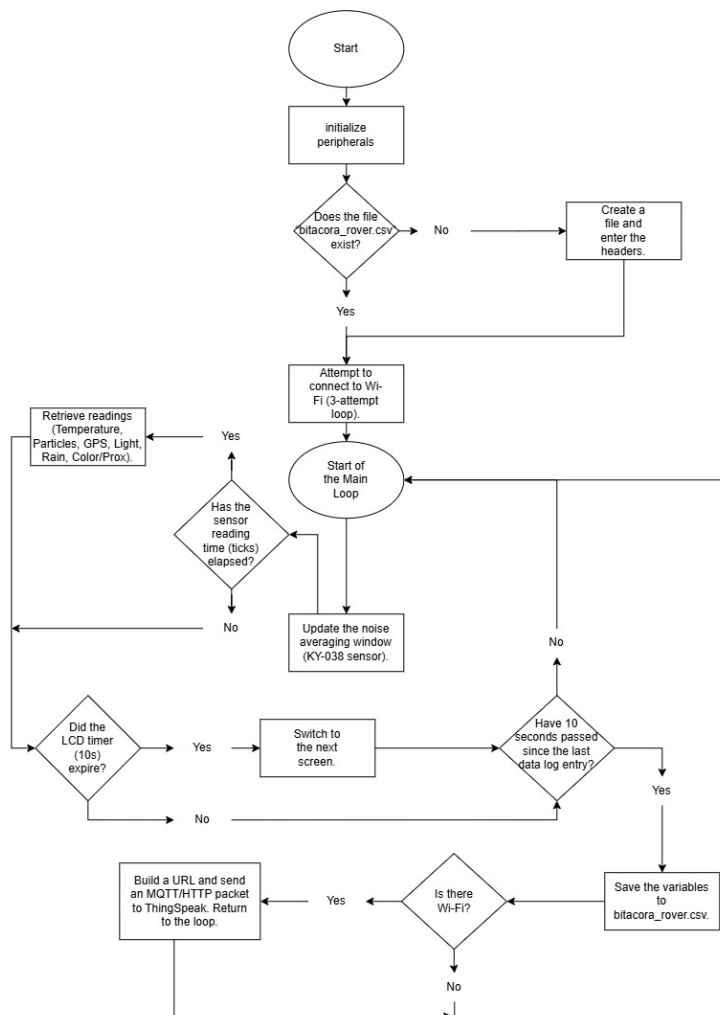


Figure 3. Execution flow of the EcoRover sensing firmware, including peripheral and local-file initialization, scheduled sensor acquisition, LCD page rotation, CSV logging, and conditional HTTP transmission to ThingSpeak.

The firmware does not apply a single sampling interval to every peripheral. Instead, acquisition and interface tasks are scheduled according to the response characteristics and intended use of each device. This prevents unnecessary consecutive requests to slower sensors while allowing continuously serviced inputs to be updated within the main loop. The timing parameters implemented in the current firmware are summarized in Table 2.

Table 2. Timing parameters and execution conditions implemented in the EcoRover sensing firmware.

Firmware task	Configured interval	Execution condition
Main-loop iteration	Approximately 0.1 s minimum delay	Coordinates sensor servicing, display management, logging, and communication tasks
DHT22 acquisition	2.5 s	Updates the stored temperature and relative-humidity values
PMS3003 acquisition	1.5 s	Updates the stored PM1.0, PM2.5, and PM10 values when a valid UART frame is available
SEN-SON-02 processing	1.0 s evaluation window	Calculates the percentage of time for which the digital acoustic signal remains active
SEN0121, TEMT6000, APDS9960, and GPS servicing	Main-loop execution	No independent fixed acquisition interval is assigned in the current firmware
LCD page rotation	10 s	Changes the active page while retaining the latest available sensor values
Local CSV record	10 s	Appends one record to the local file after the logging interval has elapsed
ThingSpeak transmission	Triggered after each logging event	Executed only when the ESP32 remains connected to Wi-Fi

The main-loop delay does not represent a calibrated sampling period because the effective execution time also depends on peripheral response and communication operations.

Acoustic activity is processed differently from the other measurements because the SEN-SON-02 provides a binary comparator output rather than a continuous sound-pressure signal. The firmware measures how long this output remains active within a one-second window and expresses the result as a relative percentage. The value is subsequently assigned to one of four descriptive categories used by the local interface: silence, low, medium, or high acoustic activity. These categories are internal indicators and should not be interpreted as measurements in decibels.

Local storage is implemented through the file `bitacora_rover.csv` in the ESP32 flash memory. During initialization, the firmware verifies whether the file exists; if it is absent, a new file and its column headers are created. If the file is already available, the program retains it and continues appending records. The logging routine stores elapsed time, temperature, relative humidity, relative acoustic activity, light response, proximity, detected color, and PM1.0, PM2.5, and PM10 values.

GPS information is acquired within the same main loop but is handled separately from the current local CSV record. The firmware processes the `$GPRMC` and `$GPGGA` NMEA sentences to obtain valid coordinates and satellite information. Latitude and longitude are retained for the remote transmission routine when a positioning solution and Wi-Fi connectivity are available.

3.2. Reactive Navigation and Obstacle Avoidance

The navigation firmware was implemented in MicroPython as an independent control routine executed by the ESP32 assigned to the mobile base. At startup, the controller configures the digital pins connected to the L298N motor driver and the trigger and echo lines of the three HC-SR04 ultrasonic sensors. The main loop then updates the front, right, and left distance measurements and selects the corresponding motor action.

The decision sequence is presented in Figure 4. The algorithm uses two distance thresholds with different purposes: the frontal threshold provides sufficient space to stop and evaluate alternative directions, whereas the lateral threshold activates shorter corrective maneuvers during forward movement.

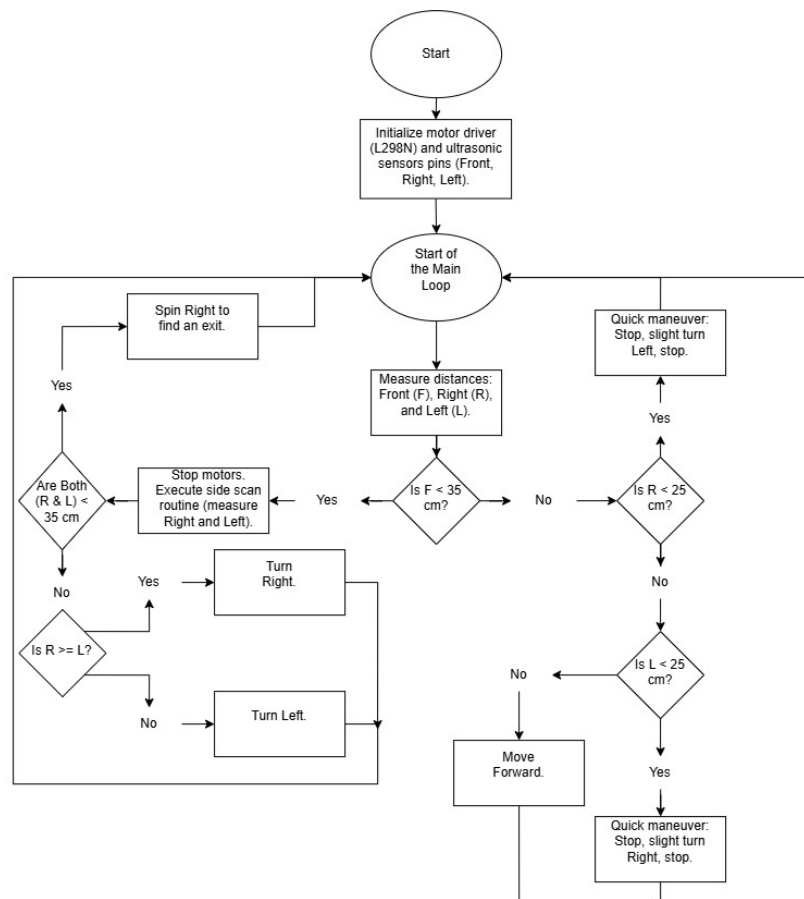


Figure 4. Flowchart of the EcoRover reactive navigation and obstacle-avoidance routine based on front, right, and left HC-SR04 distance measurements.

A frontal distance below 35 cm initiates the complete obstacle-handling routine. The motors are stopped and the rover performs timed rotations to evaluate the available space toward both sides. The direction with the greater measured clearance is then selected. If both inspected directions remain below the frontal threshold, the controller executes an additional rotation to search for an available exit.

When the frontal path remains clear, lateral distances are compared with a lower threshold of 25 cm. A short corrective maneuver is applied when an object is detected too close to either side; otherwise, the rover continues moving forward. All rotations are controlled by predefined activation times rather than by closed-loop angular-position control. After each action, the firmware returns to the main loop and acquires a new set of distance measurements.

3.3. HTTP Communication and ThingSpeak Integration

The communication between the sensing ESP32 and ThingSpeak is implemented through the platform's HTTP interface. As established in Figure 3, the transmission event is associated with the local logging interval and is executed only when the Wi-Fi interface remains connected. The firmware does not create an MQTT message or a JSON payload; instead, it constructs an HTTP GET request containing the channel write key, seven sensor fields, geographic coordinates, and a positioning-status parameter.

The correspondence between the firmware variables and the parameters included in the request is presented in Table 3. Temperature, relative humidity, relative acoustic activity, relative light response, and the three particulate-matter readings occupy fields 1–7 of the ThingSpeak channel. GPS information is sent through the dedicated latitude, longitude, and status parameters rather than through the numbered sensor fields.

Table 3. Mapping between the EcoRover firmware variables and the parameters transmitted to ThingSpeak through HTTP GET requests.

HTTP parameter	Firmware source	Transmitted information
api_key	THINGSPEAK_WRITE_KEY	Channel write authorization; the credential is not reported in the article
field1	dht['temperatura']	Ambient temperature in degrees Celsius
field2	dht['humedad']	Relative humidity in percent
field3	ruido['porcentaje']	Relative acoustic-activity percentage calculated by the firmware
field4	luz['porcentaje']	Relative light-response percentage calculated from the analog input
field5	pms['pm1']	PM _{1.0} concentration in $\mu\text{g m}^{-3}$
field6	pms['pm25']	PM _{2.5} concentration in $\mu\text{g m}^{-3}$
field7	pms['pm10']	PM ₁₀ concentration in $\mu\text{g m}^{-3}$
lat	gps['lat']	Latitude expressed in decimal degrees
long	gps['lon']	Longitude expressed in decimal degrees
status	gps['sat']	Binary indication of whether the GPS reports a valid positioning solution

The request is issued using the MicroPython urequests library. A response status code of 200 is interpreted by the firmware as a successful update, after which the connection object is closed to release memory. If Wi-Fi is unavailable or the request raises an exception, the remote update is skipped while sensor acquisition and local CSV logging continue. The current implementation does not queue failed requests for later retransmission.

The SEN0121 rain response and the APDS9960 proximity and color values are not included in the current ThingSpeak request. These variables remain available to the sensing firmware and local interface but should not be described as remotely transmitted measurements.

3.4. Web Dashboard

The EcoRover web dashboard was developed as a single-page application using JavaScript and React. The application retrieves channel information from the ThingSpeak REST API in JSON format and organizes the received data into four visualization modules: current monitoring, GPS visualization, historical consultation, and a georeferenced heatmap. The dashboard operates as a data-consultation interface and does not send movement commands or configuration parameters to the rover.

For current monitoring, the application requests the 15 most recent channel entries and refreshes the query every 15 s. The latest entry is used to update the measurement cards, while the returned sequence is used to construct the recent-data plot. The GPS module

uses the latitude, longitude, and status values returned by ThingSpeak and displays them over a Leaflet base map. Distinct coordinates received while the application is active are accumulated in the browser to produce a session trajectory; this line is not a persistent reconstruction of every route previously traveled by the rover.

The spatial visualization modules are presented in Figure 5. The heatmap allows the user to select a variable, date, and time interval. In its normal operating mode, the application requests ThingSpeak records containing valid coordinates and converts each selected measurement into a normalized intensity used by the map layer. A separate example-data mode generates synthetic coordinates and sensor values exclusively for testing the interface when suitable georeferenced records are unavailable.

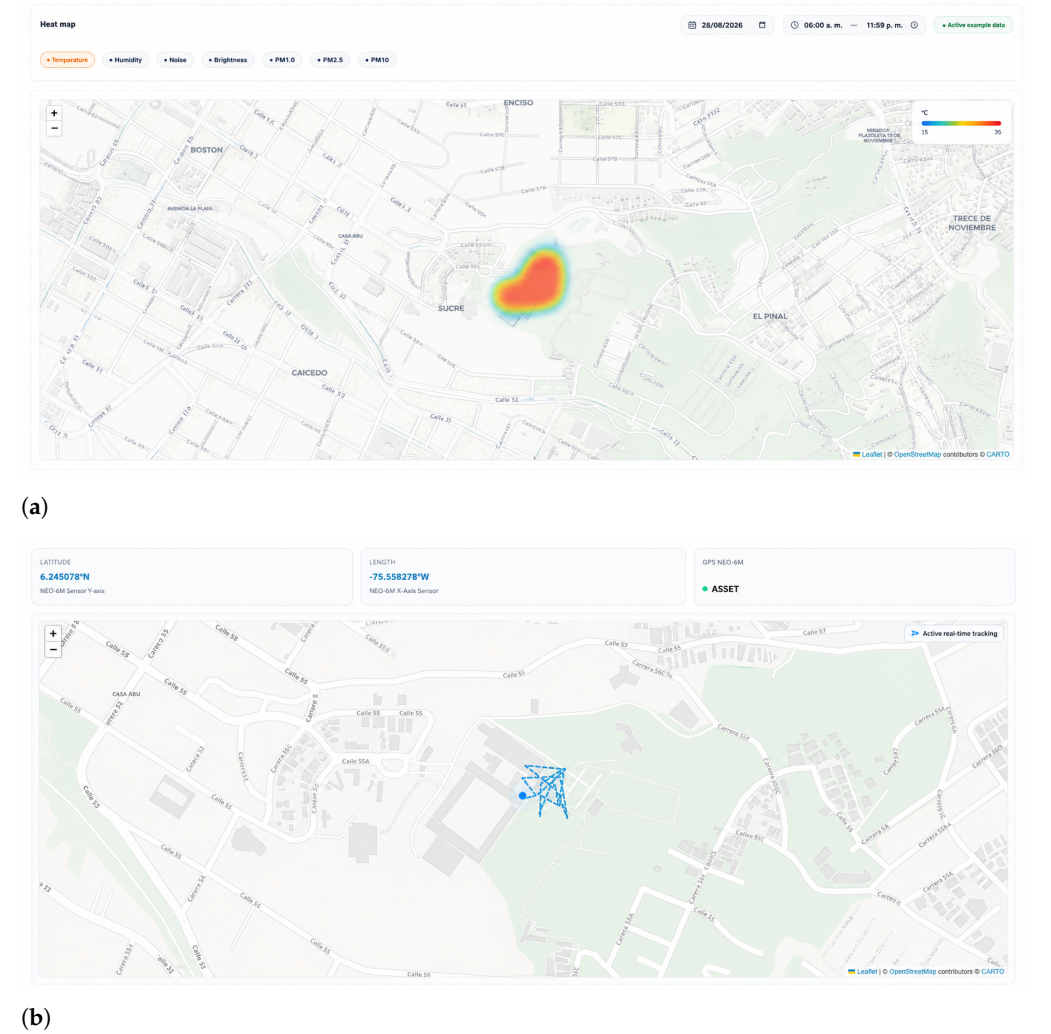
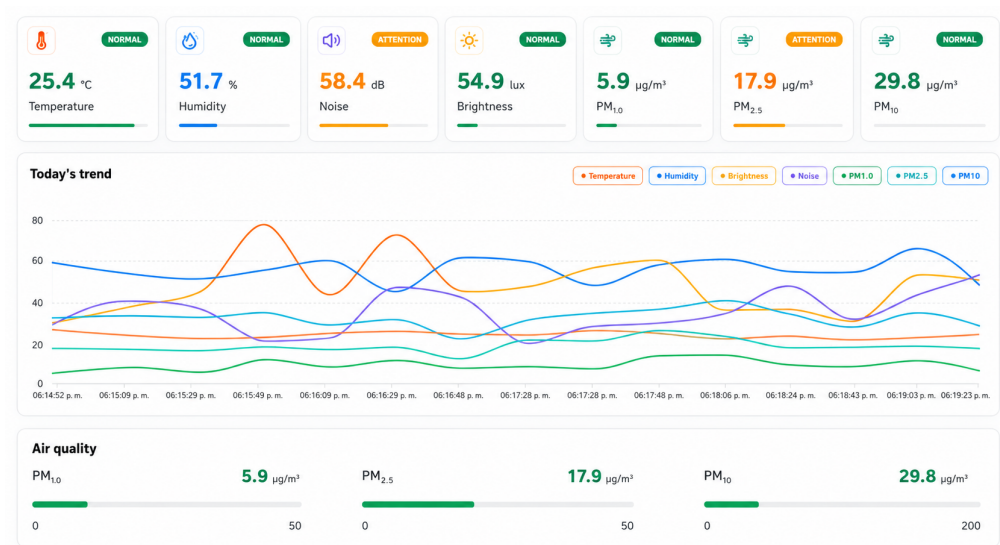
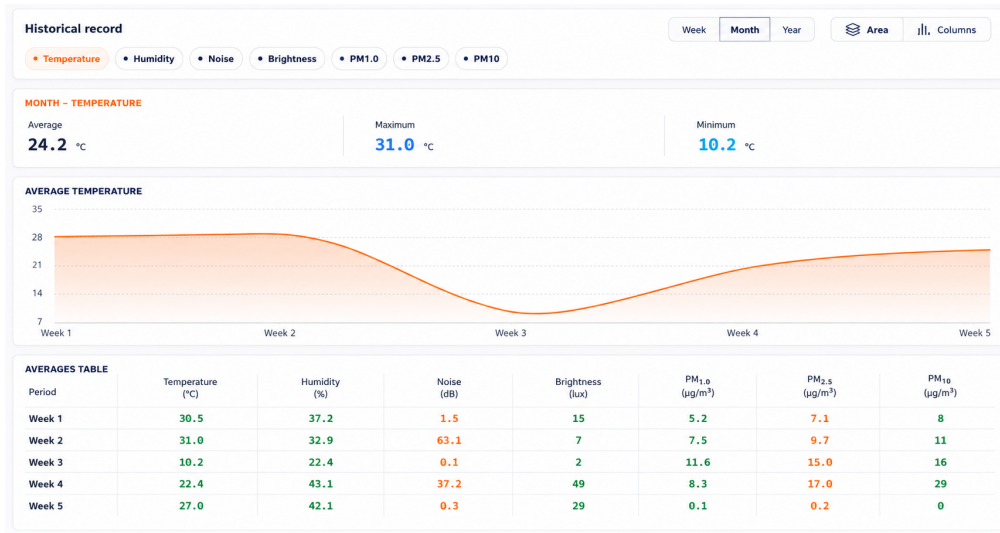


Figure 5. Spatial-visualization modules of the EcoRover web dashboard: (a) heatmap displayed in the explicitly identified example-data mode, with variable, date, and time filters; and (b) GPS view containing the latest reported coordinates, positioning status, and trajectory accumulated during the active browser session.

The historical module performs independent requests to the ThingSpeak channel according to the period selected by the user. Returned records are grouped for graphical representation, and their average, minimum, and maximum values are calculated within the web application. The user can select the variable and switch between area and column charts. Figure 6 presents the current and historical consultation modules without interpreting their displayed values as experimental results.



(a)



(b)

Figure 6. Temporal-data modules of the EcoRover web dashboard: (a) current-data view containing the latest ThingSpeak values, recent channel entries, and particulate-matter summaries; and (b) historical consultation view with period and variable selectors, summary statistics, an aggregated plot, and a grouped-value table.

4. Results and Discussion

4.1. Prototype Integration and Functional Verification

The main result of the development process was a physically integrated mobile prototype in which sensing, local visualization, data storage, wireless communication, and vehicle movement could operate within the same platform. The final assembly shown in Figure 2 was obtained through incremental component testing rather than by directly combining all the modules. This procedure was important because it identified devices that did not provide sufficiently stable operation during the initial tests. In particular, the BME680 and the evaluated KY-038 units were removed from the final configuration. Temperature and humidity acquisition was retained through the DHT22, while the SEN-SON-02 was incorporated as a digital indicator of relative acoustic activity.

The integration tests also revealed communication problems on the I2C bus shared by the APDS9960 and the LCD. These problems appeared as initialization errors and

communication timeouts when both devices were accessed repeatedly. External pull-up resistors were consequently added to the SDA and SCL lines, the physical connections were reorganized, and the firmware was modified to separate the acquisition and display-update intervals. These changes allowed both peripherals to remain integrated in the prototype. However, an error rate was not measured before and after the modifications; therefore, this result demonstrates a practical mitigation of the observed problem rather than a quantitative validation of I2C reliability.

After the individual modules and the combined firmware had been tested, the acquisition circuit was transferred from the prototyping board to a perforated phenolic board. This transition reduced the space occupied by the interconnections and allowed the sensing assembly to be fixed to the mobile chassis. The sensing node was subsequently operated together with the traction, obstacle-detection, display, storage, and communication subsystems. Particular attention was paid to changes in the supply voltage produced by motor operation because the sensing electronics and the power stage coexist on the same vehicle. Table 4 summarizes the functional outcomes obtained during component and system integration.

Table 4. Functional outcomes observed during the integration of the EcoRover prototype.

Evaluated aspect	Observed outcome	Scope of the result
Sensor selection	The retained modules produced usable outputs during individual tests. The BME680 and the evaluated KY-038 units were excluded, and the final sensing configuration used the DHT22 and SEN-SON-02 instead.	Functional selection of the sensing hardware; no metrological comparison was performed.
Multi-interface acquisition	Analog, digital, I2C, and UART peripherals were incorporated into the same MicroPython acquisition program through modular sensor classes.	Demonstrates the integration of heterogeneous peripherals in one sensing node.
Shared I2C operation	Communication faults involving the APDS9960 and LCD were mitigated through external pull-up resistors, wiring adjustments, and differentiated access intervals.	Practical improvement observed during integration; a quantitative bus-error analysis remains pending.
Local data management	The firmware implemented periodic writing to the <code>bitacora_rover.csv</code> file independently of the availability of the Wi-Fi connection.	Demonstrates offline storage capability; the file structure and inclusion of GPS information must be finalized before a monitoring campaign.
Cloud and dashboard connection	Sensor fields were transmitted to ThingSpeak through HTTP GET requests and could be retrieved by the React dashboard for temporal and spatial presentation.	Demonstrates interoperability between the embedded firmware, cloud channel, and web application.
Mobile implementation	The acquisition assembly was transferred to a perforated board, mounted on the vehicle, and operated together with the traction and navigation hardware.	Demonstrates a complete mobile proof of concept; mobility performance and route repeatability were not quantified.

4.2. Data Acquisition, Transmission, and Visualization Results

The implemented firmware completed the intended data path from sensor acquisition to local and remote presentation. Measurements from the retained modules were organized using a common key–value structure and made available to the LCD, the local data logger, and the ThingSpeak transmission routine. Local records were generated periodically even when Wi-Fi was unavailable. When a connection was present, the firmware constructed an HTTP GET request containing the seven sensor fields and the latest GPS information defined in Table 3. This behavior separates sensing from cloud availability and prevents a temporary network interruption from stopping sensor acquisition. Nevertheless, the current implementation does not queue failed transmissions for later delivery.

The web application retrieved the records stored in ThingSpeak and presented them through the four modules described in Section 3.4. The current-data and historical views shown in Figure 6 demonstrate that the application can display the latest sensor values, plot recent records, calculate descriptive statistics, group measurements by period, and switch

between the available variables. These results verify the operation of the visualization and consultation functions. They do not establish the accuracy of the values displayed because the sensors were not compared with reference instruments.

The spatial modules shown in Figure 5 also demonstrate the software mechanisms developed for associating channel records with geographic information. The GPS view can place the latest valid coordinates on a Leaflet map and connect successive positions received during the active browser session. The heatmap can filter georeferenced ThingSpeak records by variable, date, and time interval. Figure 5(a), however, was obtained using the interface's explicitly identified example-data mode. It demonstrates the operation and visual design of the heatmap but must not be interpreted as the result of an environmental mapping campaign.

The combined operation of the firmware, ThingSpeak channel, and web application therefore provides evidence of end-to-end technical integration. The defensible result at this stage is the acquisition, transfer, retrieval, and presentation of multiple sensor variables. The available evidence does not support comparisons among campus zones, identification of environmental hot spots, or classification of environmental risk.

4.3. Limitations and Improvement Priorities

The functional tests established that the proposed architecture can combine mobile robotics, heterogeneous sensing, local storage, and IoT visualization. They did not constitute a metrological validation of the platform. The DHT22 and PMS3003 measurements were not compared with traceable reference instruments, while the TMT6000, SEN-SON-02, SEN0121, and APDS9960 outputs remain relative, binary, or application-dependent indicators. Consequently, values from these devices should be used to examine system behavior and relative changes, not to make regulatory, occupational-health, or environmental-compliance assessments.

A structured campus monitoring campaign was not conducted. The current material does not contain a controlled route dataset with repeated measurements, synchronized geographic positions, reference conditions, or documented weather and operating conditions. In addition, GPS information is transmitted to ThingSpeak but is not included in the current local CSV record. This limits the reconstruction of a georeferenced route when communication is unavailable. Before environmental mapping is evaluated, the local record must include a consistent timestamp, latitude, longitude, positioning status, and all sensor fields. The heatmap should then be tested exclusively with measured records whose date, route, and acquisition conditions are documented.

The navigation subsystem provides reactive obstacle avoidance through ultrasonic distance thresholds and timed motor commands. Because the platform does not include wheel encoders, odometry, simultaneous localization and mapping, or closed-loop angular control, the vehicle cannot reproduce a planned trajectory with known positional accuracy. Navigation performance should be evaluated through repeated trials reporting obstacle-detection success, collision rate, route-completion rate, and traversal time. GPS performance should likewise be examined separately in open and semi-covered campus areas.

Communication and electrical operation also require further evaluation. The firmware should periodically attempt to restore Wi-Fi connectivity, retain unsent records for subsequent transmission, and replace the current HTTP endpoint with an encrypted HTTPS connection when supported by the embedded configuration. Separate or better-regulated power paths, additional decoupling, battery-voltage monitoring, and tests under motor start-up conditions would help determine whether the power stage affects sensor acquisition or ESP32 stability.

Finally, the dashboard must preserve the distinction between measured quantities and relative indicators. Acoustic activity should not be reported in decibels, and the TEMT6000 output should not be presented in lux until calibration functions have been established. Similarly, labels such as “normal”, “attention”, or environmental-risk categories require experimentally justified thresholds and should not be used in the present version. Once these issues are addressed, a repeatable field protocol can be designed to evaluate spatial coverage, data completeness, positioning quality, and agreement with reference instruments.

5. Conclusions

This work established EcoRover as a functional, low-cost mobile IoT prototype for exploratory environmental monitoring in smart-campus settings. Its main contribution lies in integrating mobile sensing, local data management, remote consultation, and educational robotics within a coherent and modular platform developed by students. The resulting system provides a practical basis for studying how a mobile sensing platform can complement fixed monitoring nodes. Within the scope of the available evidence, EcoRover should be understood as a functional proof of concept and as a foundation for future environmental mapping studies, rather than as a system that has already characterized campus conditions or determined environmental risk.

Progressing from the current prototype to a reproducible research platform will require comparison with reference instruments, a documented field protocol with repeated routes, complete synchronization between measurements and geographic positions, and quantitative evaluation of mobility, communication continuity, and electrical stability. These steps would allow future studies to assess spatial environmental variations while preserving a clear distinction between exploratory measurements and validated environmental information.

Supplementary Materials: The following supporting information is publicly available at <https://github.com/Jordan-R7/Eco-Rover>: Software S1, EcoRover sensor-acquisition and data-management firmware; Software S2, reactive-navigation firmware; and Software S3, web-dashboard source code.

Author Contributions: Methodology, J.D.A.R. and J.A.E.G.; software, J.D.A.R. and J.A.E.G.; validation, J.D.A.R., J.A.E.G., D.C.E.C. and H.Y.M.M.; formal analysis, D.C.E.C. and H.Y.M.M.; investigation, J.D.A.R., J.A.E.G., D.C.E.C. and H.Y.M.M.; resources, J.C.M.G. and J.A.M.C.; data curation, D.C.E.C. and H.Y.M.M.; writing—original draft preparation, J.C.M.G. and J.A.M.C.; writing—review and editing, J.C.M.G. and J.A.M.C.; visualization, J.D.A.R. and J.A.E.G.; project administration, J.C.M.G. and J.A.M.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Institución Universitaria ITM through the G8 call, grant number PG8 26101.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The records supporting the functional evaluation presented in this study are available from the corresponding author upon reasonable request. No dataset from a structured environmental mapping campaign was generated. The source code for the sensor-acquisition firmware, reactive-navigation firmware, and web dashboard is publicly accessible at <https://github.com/Jordan-R7/Eco-Rover>.

Acknowledgments: The authors thank Institución Universitaria ITM for providing the facilities and institutional support required for the development of the EcoRover prototype.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

API	Application programming interface
CSV	Comma-separated values
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
I ² C	Inter-integrated circuit
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid-crystal display
PM _{1.0}	Particulate matter with an aerodynamic diameter of up to 1.0 μm
PM _{2.5}	Particulate matter with an aerodynamic diameter of up to 2.5 μm
PM ₁₀	Particulate matter with an aerodynamic diameter of up to 10 μm
REST	Representational State Transfer
RGB	Red, green, and blue
RH	Relative humidity
SCL	Serial clock line
SDA	Serial data line
UART	Universal asynchronous receiver-transmitter

References

1. Min-Allah, N.; Alrashed, S. Smart Campus—A Sketch. *Sustainable Cities and Society* **2020**, *59*, 102231. <https://doi.org/10.1016/j.scs.2020.102231>.
2. Fortes, S.; Santoyo-Ramón, J.A.; Palacios, D.; Baena, E.; Mora-García, R.; Medina, M.; Mora, P.; Barco, R. The Campus as a Smart City: University of Málaga Environmental, Learning, and Research Approaches. *Sensors* **2019**, *19*, 1349. <https://doi.org/10.3390/s19061349>.
3. Zanella, A.; Bui, N.; Castellani, A.; Vangelista, L.; Zorzi, M. Internet of Things for Smart Cities. *IEEE Internet of Things Journal* **2014**, *1*, 22–32. <https://doi.org/10.1109/JIOT.2014.2306328>.
4. Snyder, E.G.; Watkins, T.H.; Solomon, P.A.; Thoma, E.D.; Williams, R.W.; Hagler, G.S.W.; Shelow, D.; Hindin, D.A.; Kilaru, V.J.; Preuss, P.W. The Changing Paradigm of Air Pollution Monitoring. *Environmental Science & Technology* **2013**, *47*, 11369–11377. <https://doi.org/10.1021/es4022602>.
5. Morawska, L.; Thai, P.K.; Liu, X.; Asumadu-Sakyi, A.; Ayoko, G.; Bartonova, A.; Bedini, A.; Chai, F.; Christensen, B.; Dunbabin, M.; et al. Applications of Low-Cost Sensing Technologies for Air Quality Monitoring and Exposure Assessment: How Far Have They Gone? *Environment International* **2018**, *116*, 286–299. <https://doi.org/10.1016/j.envint.2018.04.018>.
6. Dunbabin, M.; Marques, L. Robots for Environmental Monitoring: Significant Advancements and Applications. *IEEE Robotics & Automation Magazine* **2012**, *19*, 24–39. <https://doi.org/10.1109/MRA.2011.2181683>.
7. Jin, M.; Liu, S.; Schiavon, S.; Spanos, C. Automated Mobile Sensing: Towards High-Granularity Agile Indoor Environmental Quality Monitoring. *Building and Environment* **2018**, *127*, 268–276. <https://doi.org/10.1016/j.buildenv.2017.11.003>.
8. Van den Bossche, J.; Peters, J.; Verwaeren, J.; Botteldooren, D.; Theunis, J.; De Baets, B. Mobile Monitoring for Mapping Spatial Variation in Urban Air Quality: Development and Validation of a Methodology Based on an Extensive Dataset. *Atmospheric Environment* **2015**, *105*, 148–161. <https://doi.org/10.1016/j.atmosenv.2015.01.017>.
9. Kerckhoffs, J.; Hofman, J.; Khan, J.; Adams, M.D.; Blanco, M.N.; deSouza, P.; Durant, J.L.; Faridi, S.; Fruin, S.; Hankey, S.; et al. Mobile Monitoring of Air Pollution—A Position Paper on Use Cases, Good Practices, Challenges, and Opportunities. *Environment International* **2025**, *202*, 109582. <https://doi.org/10.1016/j.envint.2025.109582>.
10. Benitti, F.B.V. Exploring the Educational Potential of Robotics in Schools: A Systematic Review. *Computers & Education* **2012**, *58*, 978–988. <https://doi.org/10.1016/j.compedu.2011.10.006>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.